

Go2 Continuous Rebound Setup and Experiment Plan

1 Current Task

The current task is continuous rebound control for Go2. At reset, each environment samples a scalar target apex-height command

$$h^* \sim \mathcal{U}(0.5, 0.8) \text{ m}, \quad (1)$$

and independently samples the initial drop height

$$h_{\text{drop}} \sim \mathcal{U}(0.5, 0.8) \text{ m}. \quad (2)$$

The robot base is placed at h_{drop} with zero root velocity and the default joint state. The policy must keep rebounding for the full episode. The objective is not to jump higher forever, nor merely to recover to the initial release height, but to make each rebound apex return close to the commanded target height while maintaining a flat quadruped posture and staying in place.

Unlike the earlier single-rebound setup, reaching an apex is no longer a termination condition. The episode continues until either a failure condition is met or the 20 s time limit is reached. A successful rollout should therefore show repeated cycles of

fall, support interaction, rebound, valid apex, re-arm after the feet return near the support.

2 Environment Variants

Two environment IDs are used:

ID	Scene	Notes
Go2-Rebound-Flat	rigid plane	4096 envs, env spacing 2.5 m
Go2-Rebound-Trampoline	deformable trampoline	2048 envs, env spacing 4.0 m

The trampoline variant keeps the same rebound MDP terms but replaces the support with a deformable trampoline and repins the trampoline rim. Task success does not rely on rigid-deformable contact sensors. Apex and foot-clearance gates use only root state and kinematic foot body positions.

3 Timing

The simulator time step and action decimation are

$$\Delta t_{\text{sim}} = 0.005 \text{ s}, \quad d = 4, \quad \Delta t = d\Delta t_{\text{sim}} = 0.02 \text{ s}. \quad (3)$$

The current training episode length is

$$T_{\text{episode}} = 20 \text{ s}. \quad (4)$$

IsaacLab multiplies every weighted reward term by Δt , and logged `Episode_Reward/*` values are additionally divided by T_{episode} .

4 Reset and Command

The rebound command is

$$\mathbf{c} = [h^*]. \quad (5)$$

In the current setup, the target rebound-apex height and the initial drop height are sampled independently from the same range:

$$h^* \sim \mathcal{U}(0.5, 0.8), \quad h_{\text{drop}} \sim \mathcal{U}(0.5, 0.8). \quad (6)$$

The reset event writes h^* into the command buffer and sets the root state to

$$\mathbf{p}_{\text{root}} = \mathbf{o}_{\text{env}} + [p_x^0, p_y^0, h_{\text{drop}} + h_{\text{offset}}], \quad \mathbf{v}_{\text{root}} = \mathbf{0}, \quad (7)$$

where $h_{\text{offset}} = 0$ in the current config. Joint positions are reset to the default Go2 pose and joint velocities are reset to zero.

During the rollout, the target command is resampled after

$$t_{\text{resample}} \sim \mathcal{U}(10, 20) \text{ s}. \quad (8)$$

This changes only h^* ; it does not teleport the robot or change the latched initial drop height h_0 . With a 20 s episode, most rollouts see zero or one mid-episode target change, which is enough to test adaptation without turning the task into rapid command following.

The command term stores the following buffers:

Symbol	Buffer	Meaning
h^*	<code>_target_apex_height</code>	commanded target apex height
h_0	<code>drop_height</code>	root height latched after reset
$\hat{h}_{\text{apex}}$	<code>last_apex_height</code>	latched valid-apex root height
$\hat{h}_{\text{apex}}^*$	<code>last_apex_target_height</code>	target active at the latched valid apex
b^{apex}	<code>is_apex</code>	one-step valid-apex flag
<code>armed</code>	<code>_apex_armed</code>	next apex can be counted

5 Observations and Actions

The actor observation is

$$\mathbf{o}_t^\pi = [h^*, \mathbf{p}_t^w, \mathbf{q}_t^w, \mathbf{v}_t^b, \boldsymbol{\omega}_t^b, \tilde{\mathbf{q}}_t, \tilde{\dot{\mathbf{q}}}_t, \mathbf{a}_{t-1}]. \quad (9)$$

Here $\mathbf{p}_t^w$ is the world root position, $\mathbf{q}_t^w$ is the unique world root quaternion, $\mathbf{v}_t^b$ is base linear velocity, and $\boldsymbol{\omega}_t^b$ is base angular velocity. The actor observation uses additive noise on root position, base linear velocity, base angular velocity, joint position, and joint velocity. The critic uses the same observation terms without corruption. The policy action is a 12-DoF joint-position target action with the default Go2 joint offset. In the trampoline variant, a separate action term keeps the trampoline rim pinned.

6 Apex Event and Metrics

A valid apex is detected by a root vertical velocity sign change gated by the apex re-arm state and a geometric foot-clearance condition:

$$b_t^{\text{vel}} = (\dot{z}_{t-1} > 0) \wedge (\dot{z}_t \leq 0), \quad (10)$$

$$b_t^{\text{feet+}} = \mathbb{I} \left[\min_{i \in \mathcal{F}} z_{i,t}^{\text{local}} > z_{\text{surface}} + 0.08 \right], \quad (11)$$

where $\mathcal{F}$ is the set of four Go2 feet and $z_{\text{surface}} = 0$ in the local support frame. In the implementation this flag is named `feet_above_clearance`.

To avoid counting multiple vertical velocity zero-crossings around the same physical apex, the apex detector is armed only once per bounce. After a valid apex is counted, it is disarmed. It is re-armed only when the feet return below the same clearance threshold:

$$b_t^{\text{feet-}} = \mathbb{I} \left[\max_{i \in \mathcal{F}} z_{i,t}^{\text{local}} \leq z_{\text{surface}} + 0.08 \right]. \quad (12)$$

In the implementation this re-arm flag is named `feet_below_clearance`. Thus an apex counted by the metric/reward/timeout logic is the valid apex

$$b_t^{\text{valid}} = \text{armed}_t \wedge b_t^{\text{vel}} \wedge b_t^{\text{feet+}}. \quad (13)$$

The command term is the owner of this apex state machine. When b_t^{valid} fires, it latches the apex height and the target active at that instant as `last_apex_height` and `last_apex_target_height`. The target is latched because command resampling can occur after event detection and before the delayed reward reads the event. Reward and termination terms consume this command-owned event one manager step later; this avoids maintaining duplicate apex detectors in reward and termination code.

The logged count metrics are:

Metric	Condition	Interpretation
<code>apex_count</code>	b_t^{valid}	valid apex count
<code>height_matched_apex_count</code>	$b_t^{\text{valid}} \wedge h_t - h^* < 0.25$	apex count close to target height
<code>error_peak_height</code>	mean $ h_t - h^* $ over counted apexes	height tracking error

7 Termination

The current termination terms are:

Term	Condition	Role
<code>base_orientation</code>	base tilt angle > 0.6 rad	failure
<code>non_foot_contact</code>	non-foot rigid contact > 1	failure
<code>no_valid_apex_timeout</code>	N	failure
<code>time_out</code>	no valid apex for 2 s 20 s reached	successful long rollout if other failures absent

`no_valid_apex_timeout` reads the command-owned valid-apex pulse and resets its timer whenever that pulse is observed. This prevents a standing policy from simply waiting for the long time limit.

8 Rewards

The weighted reward is

$$R_t = \Delta t \left(-250 r_t^{\text{fail}} + 50 r_t^{\text{apex}} - 1.5 \times 10^{-2} r_t^{\text{energy}} - 2 r_t^{\text{orient}} - 0.5 r_t^{\text{inplace}} - 10^{-2} r_t^{\text{action}} - 0.05 r_t^{\text{joint}} - 10 r_t^{\text{limit}} \right). \quad (14)$$

The failure pulse is

$$r_t^{\text{fail}} = \mathbb{I}[\text{base_orientation}] + \mathbb{I}[\text{non_foot_contact}] + \mathbb{I}[\text{no_valid_apex_timeout}]. \quad (15)$$

There is no separate normal-timeout penalty in the current setup. A rollout that reaches `time_out` without triggering a failure termination is treated as a successful long rollout.

For the delayed apex pulse reward, the height error is computed from the latched command event:

$$e_t^{h,\text{apex}} = |\text{last_apex_height} - \text{last_apex_target_height}|. \quad (16)$$

The flat-body gate is

$$g_t^{\text{flat}} = \exp\left(-\frac{\|\mathbf{g}_{t,xy}^b\|^2}{0.35^2}\right), \quad (17)$$

The height reward does not include an additional soft foot-clearance gate; foot clearance is enforced only by the valid-apex event detector. The apex pulse reward is

$$r_t^{\text{apex}} = \mathbb{I}[b_{t-1}^{\text{valid}}] \exp\left(-\left(\frac{e_t^{h,\text{apex}}}{0.10}\right)^2\right) g_t^{\text{flat}}. \quad (18)$$

The remaining penalties are standard orientation, action-rate, joint-deviation, and joint-position-limit regularizers. The in-place term penalizes drift from the reset xy position and reset yaw:

$$r_t^{\text{inplace}} = \left\|\frac{\mathbf{p}_{xy,t} - \mathbf{p}_{xy,0}}{0.25}\right\|^2 + \left(\frac{\text{wrap}(\psi_t - \psi_0)}{0.5}\right)^2. \quad (19)$$

Reward scale. In the current 20 s setup, a single perfect apex pulse contributes

$$50 \cdot 0.02/20 = 0.05$$

to the logged `Episode Reward/rebound_height`. A single failure termination contributes approximately

$$-250 \cdot 0.02/20 = -0.25$$

for the environments where that failure happened, before averaging over all reset environments. With the current absolute-work weight, a typical sparse energy pulse of $W^{\text{abs}}/h^* \approx 250$ contributes

$$-1.5 \times 10^{-2} \cdot 250 \cdot 0.02/20 \approx -0.00375$$

to the logged `Episode Reward/energy_penalty` for one valid apex.

9 Implementation Files

File	Purpose
<code>go2_rebound_env_cfg.py</code>	scene, reset, reward, termination config
<code>mdp/commands.py</code>	rebound command, apex metrics, re-arm state, energy metrics
<code>mdp/events.py</code>	reset from sampled drop height
<code>mdp/rewards.py</code>	height reward, energy penalty, and failure pulses
<code>mdp/terminations.py</code>	timeout-without-apex termination
<code>scripts/rsl_rl/play-rebound.py</code>	play script with apex height and energy printout

10 Experiment Roadmap

The experiment order should separate task-definition changes from style, efficiency, and final ablations.

10.1 Step 0: Freeze the Current Continuous-Rebound Baseline

First train and save a clean baseline with the current config. Record the git state, run ID, checkpoint, and videos. The expected healthy indicators are:

- `Episode_Termination/time_out` close to 1,
- `no_valid_apex_timeout` close to 0,
- `apex_count` close to the visually observed bounce count,
- `height_matched_apex_count/apex_count` high,
- `error_peak_height` low.

10.2 Step 1: Decouple Initial Drop Height from Target Apex Height

This has been implemented in the current setup. The target height and the initial release height are no longer the same sampled scalar:

$$h^* \sim \mathcal{U}(0.5, 0.8), \quad h_{\text{drop}} \sim \mathcal{U}(0.5, 0.8).$$

The height rewards and metrics now compare apex height against h^* rather than against h_{drop} . This tests whether the policy controls rebound height, rather than only recovering to the height from which it was dropped.

10.3 Step 2: Add In-Place Hopping

This has also been implemented in the current setup with a small dense penalty on displacement from the reset xy position and reset yaw. Suggested diagnostics for the run are

$$\|\mathbf{p}_{xy,t} - \mathbf{p}_{xy,0}\|, \quad \|\mathbf{v}_{xy,t}\|, \quad |\psi_t - \psi_0|, \quad |\dot{\psi}_t|.$$

The current weight is intentionally modest so vertical rebound quality is not suppressed. The goal is to keep the robot rebounding in place without sacrificing apex-height tracking.

10.4 Step 3: Minimal Leg-Motion Regulation

Before optimizing energy, first remove obvious high-frequency or visually unacceptable leg motion. This should be a light regulation pass, not a strong posture-imitation objective. The goal is to prevent wasted actuator motion and unsafe leg swings while still allowing the robot to shape its body and legs to use the trampoline.

Test the following regularization variants one at a time:

1. increase `joint_deviation_l1` weight, e.g. $-0.05 \rightarrow -0.10$ first; avoid jumping immediately to a strong posture penalty because it can suppress useful trampoline-exploitation strategies;
2. add a small joint-velocity penalty to reduce fast flight leg motion;
3. test whether the command-side hard foot-clearance apex gate is still needed after removing the reward-side soft foot-clearance gate.

This step should be done before energy optimization so that the energy term is not merely compensating for a leg-motion artifact. After each variant, check `apex_count`, `height_matched_apex_count`, `error_peak_height`, in-place drift, and videos. If height tracking or survival degrades, the regularization is too strong.

10.5 Step 4: Add Energy Optimization on a Fixed Trampoline

Only after the behavior is stable and visually acceptable, add energy terms on the fixed trampoline. Keeping trampoline parameters fixed during this step isolates the question: can the policy maintain commanded apex height with less active work by using the trampoline’s passive energy storage?

Two mechanical-work definitions should be tested:

$$P_t^+ = \sum_j \max(\tau_{j,t} \dot{q}_{j,t}, 0), \quad P_t^{\text{abs}} = \sum_j |\tau_{j,t} \dot{q}_{j,t}|.$$

The reward is sparse. Between two valid apex events, the energy command accumulates

$$W_k^+ = \sum_{t \in k} P_t^+ \Delta t, \quad W_k^{\text{abs}} = \sum_{t \in k} P_t^{\text{abs}} \Delta t.$$

At the next valid apex, the reward penalty is normalized by the commanded target height h_k^* :

$$c_k^{\text{energy}} = \frac{W_k}{\max(h_k^*, \epsilon)}.$$

Using the target height for the reward prevents the policy from jumping higher than requested only to dilute the work-per-height penalty. The reported metrics below still use the achieved apex height, which keeps them interpretable as the actual work needed per meter of realized rebound height.

The implementation exposes this through the `mode` parameter of `energy_penalty` in `go2_rebound_env_cfg.py`. Set `mode=positive` for W_k^+/h_k^* or `mode=absolute` for W_k^{abs}/h_k^* . The raw reward follows the same convention as the apex-height pulse: Isaac Lab applies the global Δt scaling, and the event scale is handled in the configured weight. With the current $\Delta t = 0.02$ s, the current absolute-work trial weight is

$$w_{\text{energy}} = -1.5 \times 10^{-2}.$$

The positive-work term penalizes active mechanical energy injected by the motors:

$$\tau_{j,t} \dot{q}_{j,t} > 0.$$

It is the preferred first test because it discourages the robot from actively “kicking” to create apex height while still allowing negative work for landing stabilization. The absolute-work term penalizes both positive and negative mechanical work, so it also discourages active braking:

$$r_t^{\text{abs-work}} = r_t^{\text{pos-work}} + \sum_j \max(-\tau_{j,t} \dot{q}_{j,t}, 0).$$

It should be tested after the positive-work variant, or with a much smaller weight, because too much absolute-work penalty can reduce useful damping and postural stabilization during trampoline compression.

A torque-squared term,

$$\sum_j \tau_{j,t}^2,$$

is also useful as a regularizer, but it mainly discourages large torques rather than directly measuring active mechanical work.

The previous positive-work run shows the intended first-order behavior: the logged `Episode_Reward/energy_penalty` is roughly 10% of `Episode_Reward/rebound_height`, the valid apex count increases, and `positive_work_per_height` decreases. This indicates that the energy term is influencing the policy without dominating the height-tracking task.

Use a small weight or a curriculum that turns on the energy penalty only after the policy already achieves stable rebound. Keep the height and survival terms dominant. If energy decreases while `apex_count`, `height_matched_apex_count`, `error_peak_height`, or `time_out` degrades, the energy weight is too large or was introduced too early. Compare policies with positive-work-per-height and absolute-work-per-height; use braking ratio as a diagnostic. A low positive-work policy with high braking ratio may simply be absorbing trampoline energy through active braking rather than using the trampoline efficiently.

The energy command term logs:

Metric	Definition
<code>Metrics/energy/positive_work_per_height</code>	mean over valid apexes of W_k^+/h_k^{apex}
<code>Metrics/energy/absolute_work_per_height</code>	mean over valid apexes of $W_k^{\text{abs}}/h_k^{\text{apex}}$
<code>Metrics/energy/braking_ratio</code>	$W_{\text{episode}}^- / \max(W_{\text{episode}}^{\text{abs}}, \epsilon)$

10.6 Step 5: Add Trampoline Parameter Randomization

After the energy-optimized policy works on the fixed trampoline, introduce trampoline randomization. The current Go2 rebound trampoline setup uses log-uniform Young’s modulus randomization over a $0.5\times$ to $2.0\times$ stiffness range while keeping trampoline mass fixed:

$$\log E \sim \mathcal{U}(\log(4.0 \times 10^4), \log(1.6 \times 10^5)), \quad m \sim \mathcal{U}(10, 10).$$

This corresponds to $E_0 = 8.0 \times 10^4$ and $E/E_0 \in [0.5, 2.0]$. Thus the current setup is a first-stage stiffness-randomized, fixed-mass experiment.

The randomizable trampoline parameters should be introduced in stages:

Parameter	Meaning	Suggested use
<code>E / youngs_modulus</code>	membrane stiffness; controls soft versus stiff rebound timing	first-stage randomization
<code>m / mass</code>	trampoline inertia; changes response speed and energy exchange timing	first-stage randomization
<code>elasticity_damping</code> or <code>damping_scale</code>	material energy dissipation	second-stage, one damping knob at a time
<code>dynamic_friction</code>	tangential foot–trampoline friction, affecting slip and in-place stability	robustness sweep
<code>ν / poissons_ratio</code>	deformation mode and volume preservation	late-stage small-range sweep

Keep numerical and sleep parameters fixed during the main physics randomization study: `vertex_velocity_damping`, `sleep_damping`, `sleep_threshold`, `settling_threshold`, solver iteration counts, mesh resolution, `contact_offset`, and `rest_offset`. These parameters can affect stability and numerical dissipation, but they are less direct descriptions of a real trampoline than stiffness, mass, damping, friction, and Poisson ratio.

Use the current range as the first-stage domain-randomization test:

$$\log E \sim \mathcal{U}(\log(0.5E_0), \log(2.0E_0)), \quad m = m_0.$$

If this range is too difficult, narrow it temporarily, for example to $E/E_0 \in [0.8, 1.25]$, to separate learning instability from adaptation failure. If the first-stage stiffness randomization is stable, add mass randomization next, e.g. $m \sim \mathcal{U}(0.8m_0, 1.2m_0)$, where the mass term should be

used only after a smoke test confirms that runtime mass writes change trampoline dynamics. Randomizing too many parameters too early can make it difficult to tell whether a behavior change came from energy optimization, leg regulation, or trampoline dynamics. It can also encourage a more conservative active-jumping strategy that is robust but less clearly trampoline-efficient.

The randomized parameters should not be provided directly to the deployable actor policy, since the real robot does not know the true trampoline stiffness, mass, or damping. For the paper narrative, the adaptation methods should be tested in increasing order of complexity:

Method	Description	Purpose
MLP + DR	train one memoryless policy across randomized trampoline stiffness; no trampoline parameters and no history in the actor observation	baseline robust policy
MLP + history	stack recent observations/actions and feed the flattened history to an MLP	test whether short-term interaction history is enough
RNN policy	use an LSTM/GRU policy to infer hidden trampoline dynamics from longer temporal context	test recurrent online adaptation
RMA / latent estimator	train with privileged dynamics information, then learn a deployable latent estimator from history	final sim-to-real adaptation story

Privileged trampoline parameters may be used by the critic, teacher, or adaptation module during training, but not as direct actor inputs for a policy intended to transfer to hardware.

The first method is intentionally a partially observable baseline. Since the actor sees neither the trampoline parameter nor interaction history, it cannot condition its timing or leg impedance on the current stiffness. The expected failure mode is therefore an averaged strategy: performance on the soft trampoline, e.g. $E = 4.0 \times 10^4$, may improve compared with a policy trained only at $E = 8.0 \times 10^4$, but nominal-stiffness performance may degrade and the hard trampoline, e.g. $E = 1.6 \times 10^5$, may require a different timing compromise. This creates the justification for the later methods: history, recurrence, or an RMA-style latent estimator should recover stiffness-dependent behavior while keeping the deployable actor free of privileged trampoline parameters.

To make different trampoline dynamics visible in evaluation, run fixed evaluation sweeps over several discrete trampoline settings instead of relying only on random training averages. For each setting, compare height error, apex count, time-out rate, motor positive work, torque norm, contact/stance duration, and videos. A useful result should show that the policy adapts its timing, leg motion, and motor work across soft and stiff trampolines while maintaining the same commanded apex-height objective.

10.7 TODO: Real-Robot Observation Noise Measurement

Before finalizing sim-to-real observation corruption, measure the observation noise on the physical robot and use the measured ranges to set the policy observation noise. The required measurements are:

- joint position noise from the robot encoder readings;
- joint velocity noise from the robot state estimator or finite differences of encoder readings;
- IMU angular velocity noise for the base angular-velocity observation;
- mocap or state-estimator base position noise;
- mocap or state-estimator base linear-velocity noise.

For each signal, record at least a static standing segment and a representative rebound or hopping segment. Estimate bias, standard deviation, outliers, latency, and any filtering delay. The resulting noise magnitudes should be used to update the observation corruption in `go2_rebound_env_cfg.py`, especially for joint position, joint velocity, base angular velocity, base position, and base linear velocity.

10.8 Step 6: Final Ablation Experiments

Run formal ablations after the final task definition is stable. Suggested variants are:

Variant	Change	Main question
Full	final setup	reference
No apex timeout	remove <code>no_valid_apex_timeout</code>	is the anti-wait gate needed?
No foot gate	remove the command-side valid-apex foot-clearance gate	does it prevent standing or foot exploits?
No in-place reward	remove xy/yaw terms	does the policy drift?
No regulation	remove flight-leg regulation	does leg motion return?
No energy reward	remove energy term	efficiency versus performance
No trampoline DR	fix trampoline parameters	robustness versus specialization

For every variant, compare success rate, failure distribution, height error, apex count, height-matched apex count, drift metrics, energy, trampoline setting, and videos.

11 Recommended Order

The recommended order is:

1. freeze the current setup with independent drop/target heights and in-place constraints;
2. add only minimal leg-motion regulation needed to remove artifacts;
3. add energy optimization on the fixed trampoline;
4. add trampoline parameter randomization and evaluate discrete trampoline settings;
5. measure real-robot observation noise and update observation corruption;
6. run final ablations.

This order freezes the updated task definition first, removes obvious motion artifacts, optimizes active motor energy on a known trampoline, then tests robustness and adaptation across trampoline dynamics before final explanatory ablations.